

TEX File Format Specification

Format: Diablo III Texture Container (`.tex`) **Byte Order:** Little-endian **Magic:** `0xDEADBEEF` **Version:** 47

Corpus: 33,431 files analyzed

Table of Contents

- [TEX File Format Specification](#)
 - [Table of Contents](#)
 - [1. Overview](#)
 - [Pipeline Role](#)
 - [2. File Layout](#)
 - [3. SNO Preamble](#)
 - [4. Texture Descriptor](#)
 - [5. Pixel Formats](#)
 - [Data Size Calculation](#)
 - [6. Mip Table](#)
 - [Mip Count Determination](#)
 - [Mip Level Ordering](#)
 - [7. Frame / Sprite Atlas System](#)
 - [Single-Frame Files](#)
 - [Multi-Frame Atlas Files](#)
 - [8. Frame Hash Array](#)
 - [9. Frame Descriptors](#)
 - [UV Coordinate System](#)
 - [10. Pixel Data](#)
 - [Storage Order](#)
 - [Block-Compressed Data Shuffling](#)
 - [On-Disk Layout \(Per Mip\)](#)
 - [Stream Layout by Format](#)
 - [Unshuffle Example \(DXT1, 64×64 texture\)](#)
 - [Uncompressed Formats](#)
 - [Common Pixel Data Offsets](#)
 - [11. Cubemap Textures](#)
 - [Cubemap Properties](#)
 - [Mip Data Layout for Cubemaps](#)
 - [12. Texture Flags](#)
 - [13. Corpus Statistics](#)
 - [File Count and Version](#)
 - [Pixel Format Distribution](#)
 - [Dimension Distribution \(Top 20\)](#)
 - [Depth Distribution](#)
 - [Flags Distribution](#)
 - [Appendix A — Reading a TEX File \(C++\)](#)

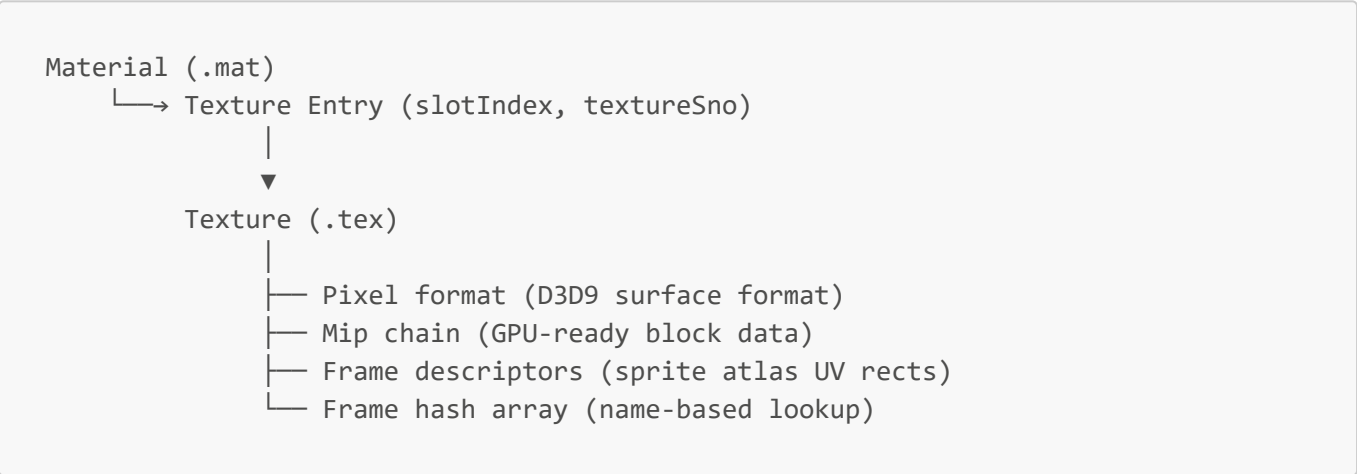
1. Overview

Texture files (**.tex**) are the primary image container for Diablo III. Each file wraps one or more Direct3D 9-compatible texture surfaces in a SNO envelope, providing:

- **Pixel data** in DXT/BC block-compressed or uncompressed ARGB formats
- **Full mip chains** (up to 10 levels, stored largest-first)
- **Sprite atlas / frame sub-images** with named UV regions and a hash lookup table
- **Cubemap support** (6-face irradiance maps stored as depth=6)

Textures are referenced by SNO ID from **.mat** material files (via **TextureEntry.textureSno**) and from **.app** appearance definitions. They are loaded by the engine's texture manager and bound to shader sampler slots.

Pipeline Role



2. File Layout

SNO Preamble	(32 bytes)
0x00: magic, version, reserved, snoId	
Texture Descriptor	(24 bytes)
0x20: pixelFormat, width, height, depth, flags, extraMipCount	
Mip Table	(480 bytes)
0x38: up to 60 × (fileOffset, dataSize) pairs	
Unused entries are zero-filled	
Atlas Metadata	(80 bytes)
0x218: frameCount, frameTableOffset, frameTableSize, reserved, sampler hints	
Hash Array SerializeData	(8 bytes)

0x268: hashArrayOffset, hashArraySize	
Frame Hash Array (frameCount × 4 bytes) 0x270: lookup indices / hash values per frame	
Frame Descriptors (frameCount × 80 bytes) At frameTableOffset: UV rects + 64-char names	
Pixel Data (mip chain, contiguous) Starts at frameTableOffset + frameTableSize Mip 0 (largest) first, descending to smallest	

Key Invariant: The first mip's file offset always equals `frameTableOffset + frameTableSize`. Validated across 10,000 files with zero mismatches.

3. SNO Preamble

Tag: TEX | **Version:** 47 | **Size:** 32 bytes

The `.tex` SNO preamble is **32 bytes** (unlike the 48-byte preamble used by `.mat` and other asset types — texture files omit the trailing `dataOffset / dataSize` pair).

```
struct SnoPreamble { // 32 bytes @ 0x00
    u32    magic;      // 0x00: Always 0xDEADBEEF
    u32    version;    // 0x04: Always 47
    u8     _reserved08[8]; // 0x08: Zero-filled
    u32    snoId;      // 0x10: Unique SNO asset identifier
    u8     _reserved14[12]; // 0x14: Zero-filled
};
```

4. Texture Descriptor

Tag: TextureDescriptor | **Version:** — | **Size:** 24 bytes

Immediately after the preamble, 24 bytes describe the texture's fundamental properties.

```
struct TextureDescriptor { // 24 bytes @ 0x20
    u32    pixelFormat; // 0x20: D3D9 surface format ID (see §5)
    u32    width;       // 0x24: Mip 0 width in pixels
    u32    height;      // 0x28: Mip 0 height in pixels
    u32    depth;       // 0x2C: 1 = 2D texture, 6 = cubemap
    u32    flags;       // 0x30: Texture flags (see §12)
    u32    extraMipCount; // 0x34: Typically mipCount - 1
};
```

extraMipCount **note:** For standard textures this field reliably equals the number of mip levels minus one. A small number of flagged textures (flags $\neq 0$) show mismatches, suggesting the engine may override the mip count when flags indicate special behavior (e.g. flipbook animations).

5. Pixel Formats

The **pixelFormat** field at 0x20 maps to Direct3D 9 surface formats. All textures in the corpus use one of nine format IDs:

ID	D3D9 Equivalent	Block Size	BPP / Block	Description	Count
0	D3DFMT_A8R8G8B8	1×1	32 bpp	Uncompressed 32-bit ARGB	277
4	D3DFMT_A4R4G4B4	1×1	16 bpp	16-bit ARGB (4444)	2
7	D3DFMT_L8	1×1	8 bpp	8-bit luminance	4
9	D3DFMT_DXT1 / BC1	4×4	8 bytes	RGB + 1-bit alpha	12,364
10	D3DFMT_DXT1 variant / BC1	4×4	8 bytes	RGB + 1-bit alpha (same as 9)	1,536
11	D3DFMT_DXT3 / BC2	4×4	16 bytes	RGBA with explicit 4-bit alpha	9,162
12	D3DFMT_DXT5 / BC3	4×4	16 bytes	RGBA with interpolated alpha (DXT5-like)	9,250
23	D3DFMT_A8	1×1	8 bpp	8-bit alpha-only	1
43	D3DFMT_ATI2 / BC5	4×4	16 bytes	Two-channel compressed (normals)	835

Data Size Calculation

For block-compressed formats (IDs 9–12, 43):

```
size = ceil(w / 4) × ceil(h / 4) × blockBytes
```

Where **blockBytes** is 8 for DXT1/BC1 (IDs 9, 10), and 16 for DXT3/BC2 (ID 11), BC3/DXT5 (ID 12), and ATI2/BC5 (ID 43).

Important: The **dataSize** in each mip table entry is the **standard interleaved** block size. On disk, block-compressed mip levels have an additional 16-byte prefix (see §10), so the actual bytes consumed in the file per mip are **dataSize + 16** for compressed formats.

For uncompressed formats:

```
size = w × h × (bpp / 8)
```

Validated with zero mismatches across the first 1,000 files in the corpus.

6. Mip Table

Tag: MipTable | **Version:** — | **Size:** 480 bytes

A fixed-size 480-byte region holds up to 60 mip-level entries as `SerializeData`-style offset/size pairs.

```
struct MipEntry {                                // 8 bytes each
    u32    fileOffset;                          // 0x00: Absolute byte offset in the file
    u32    dataSize;                            // 0x04: Size of this mip's pixel data in bytes
};

struct MipTable {                                // 480 bytes @ 0x38
    MipEntry entries[60];                      // entries[0] = largest mip (mip 0)
};                                              // Unused entries are zero-filled
```

Mip Count Determination

The active mip count is determined by scanning `entries[]` until `dataSize == 0`. Valid files have between 1 and 10 mip levels:

Mip Count	Files	Typical Dimensions
1	8,448	Any (no mip chain)
2	11	—
3	93	—
4	986	8×8 through 64×64
5	4,047	16×16 through 256×128
6	5,408	32×32 through 512×256
7	8,708	64×64 through 1024×512
8	3,742	128×128 through 2048×1024
9	1,840	256×256 through 2048×2048
10	148	512×512 through 2048×2048

Mip Level Ordering

Mip 0 (the largest / full-resolution level) is stored first. Subsequent mips are stored in descending resolution order, each exactly half the width and height of the previous level (clamped to a minimum of 1). All mip levels are stored contiguously.

7. Frame / Sprite Atlas System

Every `.tex` file contains a frame/atlas metadata block, even for non-atlas textures (which simply have `frameCount = 1`). This system enables sprite atlases — single textures containing multiple named sub-images addressed by UV rectangles.

```
struct AtlasMetadata {                                     // 80 bytes @ 0x218
    u32    frameCount;                                     // 0x218: Always ≥ 1 (1 = not an atlas)
    u32    frameTableOffset;                               // 0x21C: File offset to FrameDescriptor array
    u32    frameTableSize;                                 // 0x220: frameCount × 80 bytes
    u8     _reserved224[20];                               // 0x224: Zero-filled
    u32    field238;                                       // 0x238: Usually 0; occasionally 1, 2, 4, or 8
    u32    samplerHint1;                                   // 0x23C: Texture sampler parameter (see below)
    u32    samplerHint2;                                   // 0x240: Texture sampler parameter (see below)
    u8     _reserved244[36];                               // 0x244: Zero-filled
};
```

samplerHint1 / samplerHint2: These fields carry values that are constant across textures of all sizes and pixel formats. The dominant pair (8, 7) appears in ~93% of files. Non-default values (e.g. `samplerHint1=1`, `samplerHint2=0`) occur primarily in lightmap and single-mip textures. These likely encode engine-side texture sampler state defaults (filter mode, mip bias, or anisotropy level).

Single-Frame Files

For textures with `frameCount = 1`:

- `frameTableOffset = 0x278` (632)
- `frameTableSize = 80`
- First mip data begins at offset **0x2C8** (712)
- The single frame descriptor typically has a degenerate UV rect `(0,0,0,0)` or full `(0,0,1,1)`

This accounts for **31,969 files** (95.6% of the corpus), all sharing the same fixed offsets.

Multi-Frame Atlas Files

For atlas textures with multiple frames (e.g. inventory item icons):

- `frameTableOffset > 0x278` due to the larger hash array preceding it
- `frameTableSize = frameCount × 80`
- Frame descriptors contain named UV sub-rectangles

Example: `2DInventoryAmulet.tex` — 2048×128, 33 frames at 64×64 pixels each.

8. Frame Hash Array

Immediately before the frame descriptors, a serialized hash array provides frame lookup. Its location is given by a `SerializeData` pair at fixed offset 0x268.

```
struct FrameHashArrayHeader {                             // 8 bytes @ 0x268
    u32    hashArrayOffset;                               // 0x268: Always 0x270 (624)
```

```
    u32    hashArraySize;        // 0x26C: frameCount × 4 bytes
};

// At offset 0x270:
// u32 frameHashArray[frameCount];           // Hash / index values
```

The hash array is located at a fixed offset (0x270) regardless of frame count. For single-frame files, it is 4 bytes (one u32). For atlas textures, the array grows, which pushes the frame descriptor table to a higher offset.

Hash array contents: The first few entries in multi-frame files follow the pattern of a hash map header (bucket count, entry count, hash seed) followed by frame index values. The exact hashing scheme is not fully determined but appears to be a chained hash table enabling O(1) frame lookup by name hash.

9. Frame Descriptors

The frame descriptor array begins at `frameTableOffset` and contains `frameCount` entries, each 80 bytes.

```
struct FrameDescriptor {                                // 80 bytes each
    f32    uMin;                                         // +0x00: Left edge in normalized UV space
    f32    vMin;                                         // +0x04: Top edge in normalized UV space
    f32    uMax;                                         // +0x08: Right edge in normalized UV space
    f32    vMax;                                         // +0x0C: Bottom edge in normalized UV space
    char   name[64];                                     // +0x10: ASCII frame name, null-terminated
};
```

UV Coordinate System

UV values are normalized to the full texture dimensions:

```
pixelX = u × width
pixelY = v × height
```

Example from `2DInventoryAmulet.tex` (2048×128, 33 frames of 64×64 sub-images):

Frame	UV Rect	Pixel Rect	Name
0	(0, 0, 0, 0)	(0, 0, 0, 0)	Amulet_norm_base_01_icon
1	(0, 0, 0.0312, 0.5)	(0, 0, 64, 64)	Amulet_norm_base_02_icon
2	(0.0312, 0, 0.0625, 0.5)	(64, 0, 128, 64)	Amulet_norm_base_03_icon
3	(0.0625, 0, 0.0938, 0.5)	(128, 0, 192, 64)	Amulet_norm_base_04_icon

Frame 0 placeholder: In atlas textures, frame index 0 often has a degenerate UV rect (0, 0, 0, 0) while still carrying a valid name. This appears to be a sentinel / default entry; the engine may skip it during atlas lookups.

10. Pixel Data

Pixel data begins immediately after the frame descriptor array and contains all mip levels stored contiguously.

```
pixelDataOffset = frameTableOffset + frameTableSize
```

This is confirmed across 10,000 sampled files with zero exceptions. The first mip entry's `fileOffset` always equals this computed value.

Storage Order

Mip levels are stored **largest first** (mip 0, mip 1, ..., mip N), with each level's data contiguous. For block-compressed formats, data is **not** stored in standard interleaved DXT/BC block order — it uses a shuffled (planar) layout described below.

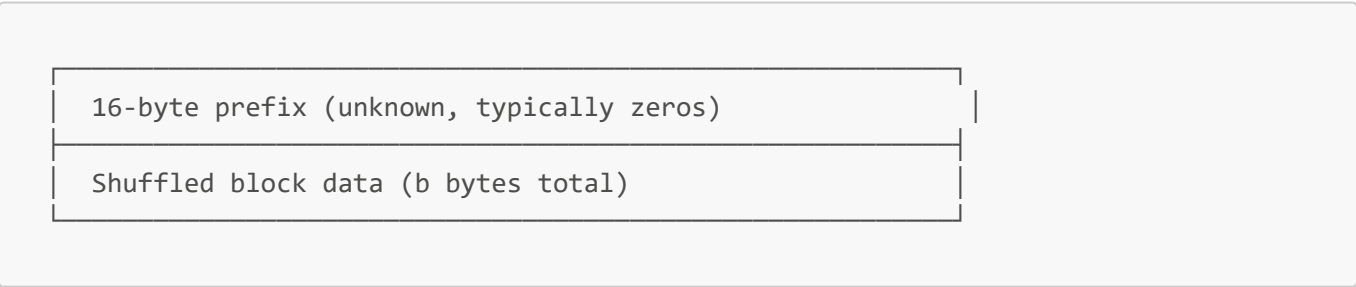
Block-Compressed Data Shuffling

Block-compressed mip levels (format IDs 9, 10, 11, 12, 43) are stored in a **shuffled / planar layout** rather than the standard interleaved block order expected by Direct3D. Each mip level has a **16-byte prefix** of unknown purpose followed by the block component streams laid out contiguously.

To convert the on-disk data to standard DXT blocks, you must **unshuffle** (deinterleave) the component streams back into per-block order. To write a TEX file, you must **shuffle** (split) standard blocks into separate streams.

On-Disk Layout (Per Mip)

For a mip level with standard interleaved size `b` bytes, let $y = b / 2$:



Total on-disk size per mip = $16 + b$.

Stream Layout by Format

DXT1 / BC1 (IDs 9, 10) — 8 bytes/block:

```
block_count = y / 4

Offset range      Stream      Size per block
```

```
[0,    y)           Color values (cv)    4 bytes  (2× u16 color endpoints)
[y,    2y)          Color indices (cp)   4 bytes  (4×4 pixel, 2-bit lookups)

Standard block = [ cv[i] | cp[i] ] (8 bytes)
```

DXT3 / BC2 (ID 11) — 16 bytes/block:

```
block_count = y / 8

Offset range      Stream              Size per block
-----
[0,    y)         Alpha pixels (ap)    8 bytes  (4×4 explicit 4-bit alpha)
[y,    y+y/2)     Color values (cv)    4 bytes  (2× u16 color endpoints)
[y+y/2, 2y)       Color indices (cp)   4 bytes  (4×4 pixel, 2-bit lookups)

Standard block = [ ap[i] | cv[i] | cp[i] ] (16 bytes)
```

DXT5 / BC3 (ID 12) and ATI2 / BC5 (ID 43) — 16 bytes/block:

```
block_count = y / 8

Offset range      Stream              Size per block
-----
[0,    y/4)       Alpha values (av)    2 bytes  (2× u8 alpha endpoints)
[y/4,  y)         Alpha indices (ap)    6 bytes  (4×4 pixel, 3-bit lookups)
[y,    y+y/2)     Color values (cv)    4 bytes  (2× u16 color endpoints)
[y+y/2, 2y)       Color indices (cp)   4 bytes  (4×4 pixel, 2-bit lookups)

Standard block = [ av[i] | ap[i] | cv[i] | cp[i] ] (16 bytes)
```

Unshuffle Example (DXT1, 64×64 texture)

```
blocks_x  = ceil(64 / 4) = 16
blocks_y  = ceil(64 / 4) = 16
block_count = 256
b         = 256 × 8      = 2048 bytes (standard size)
y         = 1024

On-disk mip data (after 16-byte prefix):
[ 0..1023 ] color values  - 256 blocks × 4 bytes
[1024..2047] color indices - 256 blocks × 4 bytes

Reassembled standard DXT1 blocks:
for i in 0..255:
    block[i] = cv[i*4..i*4+3] + cp[i*4..i*4+3]    // 8 bytes
```

Uncompressed Formats

Formats 0 (A8R8G8B8), 4 (A4R4G4B4), 7 (L8), and 23 (A8) store pixel data directly without any shuffling or prefix. The mip table offset points directly to raw pixel data.

Common Pixel Data Offsets

Offset	Frame Count	Files
0x2C8 (712)	1	31,969
0x3C0 (960)	4–8	418
0x318 (792)	2	223
0x370 (880)	3	99
Other	>8	722

11. Cubemap Textures

Cubemap textures are identified by **depth = 6**, representing the six faces of a cube map. The corpus contains **34 cubemap files**, all used as irradiance environment maps.

Cubemap Properties

All observed cubemaps share these characteristics:

- Pixel format: DXT1/BC1 (ID 9) or BC3 variant (ID 12)
- Dimensions: 32×32, 64×64, or 128×128
- Flags: always 0
- Used for image-based lighting (irradiance probes)

Mip Data Layout for Cubemaps

Mip table entries for cubemaps store the **per-face data size**, not the total for all 6 faces:

```
mipEntry.dataSize = singleFaceSize(w, h, format)
totalMipData      = mipEntry.dataSize × depth
```

Example: **BattleNet_MainScreen_Irradiance.tex** — 64×64 DXT1, depth=6:

- Mip 0 entry size: 2,048 bytes = single 64×64 DXT1 face
- Expected 6-face total: 12,288 bytes
- The engine reads **size × 6** bytes per mip level

Face ordering follows the standard Direct3D cubemap convention: **+X, -X, +Y, -Y, +Z, -Z**

12. Texture Flags

The **flags** field at offset 0x30 is a bitfield controlling special texture behavior.

Bit	Mask	Name (Tentative)	Files	Description
0	0x01	FLIPBOOK	640	Flipbook / animated texture; may override mip count
1	0x02	TILEABLE	110	Texture wraps / tiles seamlessly
0+1	0x03	FLIPBOOK + TILEABLE	6	Both flags combined
2	0x04	FLAG_04	1	Rare; purpose unknown
1+2	0x06	TILEABLE + FLAG_04	1	Rare; purpose unknown

97.7% of files (32,673) have **flags** = 0. Flagged textures may exhibit **extraMipCount** values that don't match the actual mip table entry count, as the engine may use different mip chain behavior for animated or special textures.

13. Corpus Statistics

File Count and Version

- **Total files:** 33,431
- **SNO version:** 47 (100% of files)
- **File sizes:** 736 bytes (minimum) to 22.37 MB (maximum)

Pixel Format Distribution

Format	ID	Files	Percentage
DXT1 / BC1	9	12,364	37.0%
DXT5 / BC3	12	9,250	27.7%
DXT3 / BC2	11	9,162	27.4%
DXT1 variant / BC1	10	1,536	4.6%
ATI2 / BC5	43	835	2.5%
A8R8G8B8	0	277	0.8%
L8	7	4	<0.1%
A4R4G4B4	4	2	<0.1%
A8	23	1	<0.1%

Block-compressed formats (DXT/BC) account for **99.2%** of all texture files.

Dimension Distribution (Top 20)

Dimensions	Files	Dimensions	Files
------------	-------	------------	-------

Dimensions	Files	Dimensions	Files
256×256	8,373	2048×1024	573
128×128	4,595	256×128	556
512×512	3,754	128×64	287
512×256	3,320	1024×512	268
64×64	3,316	256×512	266
1024×1024	2,040	64×128	257
2048×2048	1,439	512×128	249
128×256	1,281	1024×2048	205
32×32	799	64×256	198
2048×512	176	256×64	171

Depth Distribution

Depth	Meaning	Files
1	2D texture	33,397
6	Cubemap	34

Flags Distribution

Flags	Files	Percentage
0x00	32,673	97.7%
0x01	640	1.9%
0x02	110	0.3%
0x03	6	<0.1%
0x04	1	<0.1%
0x06	1	<0.1%

Appendix A — Reading a TEX File (C++)

```
FILE* f = fopen("texture.tex", "rb");

// — §3  SNO Preamble
SnoPreamble preamble;
fread(&preamble, sizeof(SnoPreamble), 1, f);
assert(preamble.magic == 0xDEADBEEF);
assert(preamble.version == 47);
```

```

// — §4 Texture Descriptor —————
TextureDescriptor desc;
fread(&desc, sizeof(TextureDescriptor), 1, f);

printf("Format=%u %ux%u depth=%u flags=0x%X\n",
       desc.pixelFormat, desc.width, desc.height,
       desc.depth, desc.flags);

// — §6 Mip Table —————
fseek(f, 0x38, SEEK_SET);
MipEntry mips[60];
fread(mips, sizeof(MipEntry), 60, f);

u32 mipCount = 0;
for (u32 i = 0; i < 60; i++) {
    if (mips[i].dataSize == 0) break;
    mipCount++;
}
printf("Mip levels: %u\n", mipCount);

// — §7 Atlas Metadata —————
fseek(f, 0x218, SEEK_SET);
AtlasMetadata atlas;
fread(&atlas, sizeof(AtlasMetadata), 1, f);
printf("Frames: %u\n", atlas.frameCount);

// — §9 Frame Descriptors —————
fseek(f, atlas.frameTableOffset, SEEK_SET);
for (u32 i = 0; i < atlas.frameCount; i++) {
    FrameDescriptor frame;
    fread(&frame, sizeof(FrameDescriptor), 1, f);
    printf(" [%u] UV=(%.4f,%.4f,%.4f,%.4f) '%s'\n",
           i, frame.uMin, frame.vMin, frame.uMax, frame.vMax, frame.name);
}

// — §10 Read Mip 0 pixel data —————
u32 pixelDataOffset = atlas.frameTableOffset + atlas.frameTableSize;
fseek(f, pixelDataOffset, SEEK_SET);

bool isBC = (desc.pixelFormat >= 9 && desc.pixelFormat <= 12)
            || desc.pixelFormat == 43;

if (isBC) {
    // Block-compressed: skip 16-byte prefix, read shuffled data, unshuffle
    u8 prefix[16];
    fread(prefix, 1, 16, f); // 16-byte prefix (skip)

    std::vector<u8> shuffled(mips[0].dataSize);
    fread(shuffled.data(), 1, mips[0].dataSize, f);

    // Unshuffle into standard DXT block order (see §10)
    std::vector<u8> mip0(mips[0].dataSize);
    unshuffle_bc_blocks(desc.pixelFormat, shuffled.data(),

```

```

        mip0.data(), mips[0].dataSize);
    } else {
        // Uncompressed: read directly
        std::vector<u8> mip0(mips[0].dataSize);
        fread(mip0.data(), 1, mips[0].dataSize, f);
    }

    // For cubemaps (depth=6): each face has its own 16-byte prefix + shuffled data
    if (desc.depth == 6) {
        u32 faceSize = mips[0].dataSize;
        u32 faceDiskSize = faceSize + (isBC ? 16 : 0);
        fseek(f, pixelDataOffset, SEEK_SET);
        for (u32 face = 0; face < 6; face++) {
            if (isBC) {
                u8 prefix[16];
                fread(prefix, 1, 16, f);
            }
            std::vector<u8> faceData(faceSize);
            fread(faceData.data(), 1, faceSize, f);
            if (isBC) {
                std::vector<u8> unshuffled(faceSize);
                unshuffle_bc_blocks(desc.pixelFormat, faceData.data(),
                                   unshuffled.data(), faceSize);
                // unshuffled now contains standard DXT blocks for this face
            }
        }
    }

    fclose(f);

```

Appendix B — All Structures Summary

```

// §3 – SNO Preamble (texture variant, 32 bytes)
struct SnoPreamble { // 32 bytes
    u32 magic; // 0xDEADBEEF
    u32 version; // 47
    u8 _reserved08[8];
    u32 snoId;
    u8 _reserved14[12];
};

// §4 – Texture Descriptor
struct TextureDescriptor { // 24 bytes
    u32 pixelFormat; // D3D9 format ID (see §5)
    u32 width; // Mip 0 width
    u32 height; // Mip 0 height
    u32 depth; // 1=2D, 6=cubemap
    u32 flags; // Bitfield (see §12)
    u32 extraMipCount; // Typically mipCount - 1
};

```

```
// §6 – Mip Table Entry
struct MipEntry { // 8 bytes
    u32 fileOffset; // Absolute offset in file
    u32 dataSize; // Byte size (per-face for cubemaps)
};

// §7 – Atlas Metadata
struct AtlasMetadata { // 80 bytes
    u32 frameCount; // ≥ 1
    u32 frameTableOffset; // File offset to FrameDescriptor[]
    u32 frameTableSize; // frameCount × 80
    u8 _reserved224[20];
    u32 field238; // Usually 0
    u32 samplerHint1; // Usually 8
    u32 samplerHint2; // Usually 7
    u8 _reserved244[36];
};

// §8 – Frame Hash Array Header
struct FrameHashArrayHeader { // 8 bytes
    u32 hashArrayOffset; // Always 0x270
    u32 hashArraySize; // frameCount × 4
};

// §9 – Frame Descriptor
struct FrameDescriptor { // 80 bytes
    f32 uMin; // Left UV (0.0–1.0)
    f32 vMin; // Top UV (0.0–1.0)
    f32 uMax; // Right UV (0.0–1.0)
    f32 vMax; // Bottom UV (0.0–1.0)
    char name[64]; // Null-terminated ASCII
};
```

Specification derived from binary analysis of 33,431 TEX files from Diablo III: Reaper of Souls.